Problem Statement

1. Project Overview

Build **Sahayak**, a Chrome side-panel AI agent that can understand the page currently open in the browser and help the user act on it safely.

Sahayak should be able to:

- Summarise articles and **public YouTube videos** in English or Hinglish.
- Chat with the current web page and answer only from the page context.
- Explain, translate, or draft replies from selected page content.
- Read visible screen content through screenshot-based questions.
- Detect interactive elements such as buttons, links, inputs, dropdowns, and checkboxes.
- Turn those elements into a numbered action map that an AI model can reason about.
- Execute browser actions only after the user approves each step.
- Remember a local user profile and reuse it for forms, drafts, and saved one-click skills.
- Apply safety rules that prevent risky actions on sensitive pages or fields.
- Run locally as a Chrome extension using free-tier tools wherever possible.

The project must be completed in **4 weeks** and should result in a safe, installable Chrome extension that can be shared through a public GitHub release.

---

2. Problem Statement

Most AI assistants can explain information, but they are separated from the browser context where users actually work. Users still need to copy text into a chatbot, explain what page they are viewing, identify which field or button matters, and manually carry out every browser step.

More capable browser agents can automate these tasks, but they may be paid, unavailable to some users, or too autonomous for actions that involve personal information or consequential clicks.

The goal of Sahayak is to create a **user-controlled browser agent** that combines understanding and action while keeping the user in charge.

Sahayak must understand the current page, reason about visible and interactive content, propose one action at a time, and wait for explicit approval before acting. It must also include privacy and safety restrictions so it does not operate on sensitive pages or enter protected information.

---

3. Primary Goal

At the end of four weeks, a user should be able to open Sahayak in the Chrome side panel on a supported page and do the following:

1. Ask Sahayak to summarise the current article or a public YouTube link.

2. Switch the response language between English and Hinglish.
3. Ask follow-up questions about the page.
4. Use quick actions such as Explain, Translate, and Draft a reply.
5. Ask questions about visible non-text content using a screenshot.
6. Give Sahayak a browser task such as filling a practice form or finding information.
7. Review the next proposed action and choose **Approve**, **Skip**, or **Stop**.
8. Let Sahayak perform only the approved browser action.
9. See a log of the actions that were proposed and performed.
10. Reuse locally stored profile information and saved skills.

Sahayak is not intended to be a fully autonomous browser bot. It is an **agent the user controls**.

---

4. Target Platform and Core Stack

Browser

- Google Chrome on desktop.
- Chrome Extension Manifest V3.
- Chrome Side Panel API.
- Chrome content scripts and `chrome.scripting` for page interaction.
- `chrome.tabs.captureVisibleTab` for screen capture.
- `chrome.contextMenus` for right-click actions.
- `chrome.storage.local` for local memory and preferences.

Development Environment

- **Cursor** as the primary code editor and AI-assisted development environment.

AI and Content Processing

- Gemini Flash-Lite as the default everyday model.
- Gemini Flash for harder image or reasoning tasks when needed.
- Groq free tier as a text-only fallback when appropriate.
- Gemini public YouTube link input for supported public YouTube content.
- Mozilla Readability.js for extracting readable article content.

Distribution

- GitHub repository.
- GitHub Releases for an installable extension ZIP.
- GitHub Pages for a simple project landing page.

---

5. Four-Week Build Plan

Week 1 - The Eyes: Read the Web

Objective

Create the Chrome extension foundation and give Sahayak the ability to read and summarise web content.

Required Features

1. Chrome Extension Foundation

Create a Manifest V3 Chrome extension containing:

- Extension manifest.
- Side-panel interface.
- Background service worker.
- Settings area for the user's Gemini API key.
- Ability to load the project through Chrome's **Load unpacked** flow.

2. First AI Interaction

The user should be able to select text on a page and ask Sahayak to explain it in plain language.

The result should be displayed inside the side panel.

3. Article Summarisation

Use Readability.js to extract the useful article content while excluding navigation, advertisements, footers, and other page noise.

Generate a structured summary containing:

- One-line TL;DR.
- Five key points.
- What to do next.

4. Public YouTube Summarisation

For a supported public YouTube link, Sahayak should provide a useful structured summary.

Where supported, the result should include chapter-style notes with clickable timestamps that jump to the relevant moment.

Private or unsupported YouTube content is outside the required scope.

5. Language Toggle

The user must be able to switch output between:

- English.
- Hinglish.

Week 1 Acceptance Criteria

Week 1 is complete when:

- The extension loads successfully in Chrome.
- The side panel opens next to a normal webpage.
- The API key can be entered through extension settings rather than hard-coded.
- Selected text can be explained by the AI.
- A normal article can be summarised into the required structure.
- A supported public YouTube link can be summarised.
- English/Hinglish switching works.

Optional Stretch Goal

If the device supports Chrome's built-in AI requirements, add on-device/offline article summarisation as an optional enhancement. This is not required for the core build.

Week 2 - The Brain: Understand the Page and the Screen

Objective

Turn the side panel into a context-aware assistant that can reason about the current page and visible screen content.

Required Features

1. Page Chat

Add a chat interface that uses the current page as context.

The assistant must:

- Answer questions using the page content.
- Avoid inventing information that is not present on the page.
- Clearly say when the answer is not available from the current page.
- Keep conversation history for the active interaction.
- Stream responses into the side panel when supported.

2. Right-Click AI Actions

Add Chrome context-menu actions for selected text:

- **Explain this** - simplify jargon or dense text.
- **Translate** - translate into Hindi or another selected language.
- **Draft a reply** - create a reply for content such as an email, LinkedIn post, or forum message.

All results should open in or be sent to the Sahayak side panel.

3. Screenshot-and-Ask

Allow the user to capture the visible browser tab and ask a question about visual content such as:

- Charts.
- Diagrams.
- Error messages.
- Forms.
- Other visible content that cannot be understood from extracted text alone.

The user should be able to focus the request on the relevant part of the screen before it is sent to the model.

4. Free-Tier Resilience

Sahayak must fail gracefully when model limits are reached.

Required behaviour:

- Detect rate-limit responses such as HTTP 429.
- Wait and retry where appropriate.
- Use the lighter model by default.
- Switch to a stronger Gemini model only when needed.
- Use a text-only fallback provider for eligible text tasks.
- Show a helpful error state instead of crashing.

Week 2 Acceptance Criteria

Week 2 is complete when:

- The user can chat with the current page.
- Answers remain grounded in page content.
- Explain, Translate, and Draft a reply work from the right-click menu.
- The user can ask a question about a visible screenshot.
- Model errors and rate limits do not crash the extension.

Week 3 - The Hands: Act With User Approval

Objective

Give Sahayak controlled computer-use capabilities without allowing it to act autonomously.

Required Features

1. Interactive Element Scanner

Create a content script that scans the current page for interactive elements including:

- Buttons.
- Links.
- Text inputs.
- Text areas.
- Dropdowns.
- Checkboxes.
- Other supported user-action elements.

Assign each detected element a stable number for the current page state.

2. Numbered Action Map

Display numbered badges over detected interactive elements.

Create a compact structured representation of those elements for the AI so the model reasons over a concise action map instead of raw page HTML.

3. Find It for Me

Allow the user to ask a navigation question such as:

Where do I upload my resume?

Sahayak should identify the most relevant numbered element and highlight it without taking the action automatically.

The model response for element selection should use strict structured JSON.

4. Agent Loop

Implement a one-step-at-a-time agent loop:

1. Read the current page state.
2. Build or refresh the action map.
3. Ask the AI for the **single next action**.
4. Receive that action as strict JSON.
5. Show the proposed action and reason to the user.
6. Wait for the user to choose **Approve**, **Skip**, or **Stop**.
7. Execute only an approved action.
8. Read the updated page state.
9. Repeat until the task is complete or stopped.

Supported action types should include:

- `click`
- `type`
- `select`
- `scroll`
- `done`

Each proposed action should include at least:

- Action type.
- Target element number when applicable.
- Required value when applicable.
- One-line reason.

5. Agent Controls

The agent must include:

- Maximum step limit.
- Stop button.
- Skip action.
- Dry-run mode that highlights the intended element without acting.
- Progress persistence so a paused approval flow is not lost when the extension background worker sleeps.
- Action log containing the proposed step, reason, and user decision.

6. Practice Tasks

The agent should be capable of completing controlled practice tasks such as:

- Filling a multi-step internship application from a short instruction.
- Searching a practice shop and applying filters.
- Collecting information from multiple pages into a table.

Week 3 Acceptance Criteria

Week 3 is complete when:

- Interactive elements are detected and numbered reliably on supported pages.
- Sahayak can identify the correct element from a natural-language request.
- The next action is returned in strict JSON.
- No browser action happens before user approval.
- Approve, Skip, Stop, step limit, and dry-run controls work.
- The agent survives normal service-worker sleep by saving progress.
- An action log is preserved for the task.
- Sahayak can complete at least two supported practice errands with approval at every step.

Week 4 - Memory, Safety, and Shipping

Objective

Make Sahayak personal, reusable, safe, and installable by other users.

Required Features

1. Local Profile Memory

Create a user profile stored only on the user's machine using `chrome.storage.local`.

The profile may contain fields such as:

- Name.
- College.
- Skills.
- Links.
- Preferred language.
- Preferred tone.

Use this profile when relevant for:

- Filling supported forms.
- Drafting replies.
- Personalising common workflows.

2. One-Click Skills

Allow the user to save reusable prompt recipes that can run on the current page.

Example skills from the cohort brief include:

- Exam notes.
- Explain in Hinglish.
- Reply to this email politely.
- Fill this form from my profile.

Skills should be exportable and importable as a small local file.

3. Safety Rails

Sahayak must include explicit safety restrictions.

Required rules:

- Block agent actions on banking, payment, and UPI pages.
- Never type into password fields.
- Never type into OTP fields.
- Never type card information.
- Require an additional confirmation before consequential buttons such as **Submit**, **Pay**, or **Send**.
- Provide an immediate kill switch / Stop control.
- Keep API keys out of the public GitHub repository.
- Do not send private or sensitive pages to the free-tier model.
- Restrict agent errands to the provided practice environment or pages the user owns/controls.
- Do not automate logins, payments, or OTP workflows.

4. Public Packaging

Prepare Sahayak so another user can install and understand it.

Required outputs:

- Public GitHub repository.
- Clean README.
- Setup instructions.
- Screenshots showing the extension and its key features.
- GitHub Release ZIP that can be installed with **Load unpacked**.
- One-page GitHub Pages landing site.

Week 4 Acceptance Criteria

Week 4 is complete when:

- User profile memory works locally.
- Saved skills can be created and reused.
- Skills can be exported and imported.
- Restricted pages and protected fields are blocked.
- Consequential actions require extra confirmation.
- The API key is not committed to the repository.
- The project can be installed by another user from the release package.
- The repository contains clear setup and usage documentation.
- A simple public landing page exists.

---

6. Functional Requirements

FR-1: Side Panel

Sahayak must run primarily inside the Chrome side panel and remain available while the user works on the current page.

FR-2: Settings

The extension must provide a settings interface for API credentials and user preferences.

FR-3: Content Extraction

Sahayak must distinguish readable article content from page chrome and irrelevant layout content.

FR-4: Summarisation

Sahayak must produce structured summaries and support English/Hinglish output.

FR-5: Grounded Page Chat

Page chat must use the current page as its source of truth and admit when the page does not contain the requested answer.

FR-6: Visual Understanding

The user must be able to ask questions about the visible browser screen when text extraction is insufficient.

FR-7: Structured Agent Decisions

Agent decisions must be returned in strict machine-readable JSON so application code controls execution.

FR-8: Human Approval

Every agent action must require explicit user approval before execution.

FR-9: Local Memory

Personal profile information and preferences must be stored locally using Chrome storage.

FR-10: Safety Enforcement in Code

Safety rules must be enforced by extension code, not only by model instructions.

---

7. Non-Functional Requirements

Reliability

- Handle API failures and rate limits without crashing.
- Preserve agent progress across service-worker suspension where needed.
- Validate model-generated JSON before using it.

Privacy

- Keep API keys out of source control.
- Keep profile memory on the local device.
- Avoid sending private or sensitive pages to external models.

Safety

- User approval is mandatory for actions.
- Sensitive pages and fields must be blocked by deterministic code.
- The user must always have a visible way to stop an active task.

Cost

- The core project should work with the free stack described in the cohort plan.
- Stronger or paid model tiers are optional upgrades, not requirements for the core project.

Usability

- Main features should be reachable from the side panel.
- Errors should be understandable to a non-expert user.
- The extension should make the proposed browser action clear before asking for approval.

---

8. Out of Scope

The following are not required for the four-week core build:

- Chrome on Android or iPhone.
- Autonomous execution without user approval.
- Banking or payment automation.
- Login automation.
- OTP automation.
- Entering passwords or card information.
- Sending private email, banking, or personal-document pages to an external free-tier AI service.
- Acting on arbitrary third-party sites for risky or consequential workflows.
- Chrome Web Store publication; a GitHub release installed through **Load unpacked** is sufficient for the core build.

---

9. Definition of Done

Sahayak is considered complete when a new user can install the extension from the project release, enter their own API key, open the side panel on a supported webpage, and successfully use the complete workflow below:

1. Summarise an article or supported public YouTube link.
2. Switch between English and Hinglish.
3. Chat with the current page.
4. Use Explain, Translate, or Draft a reply from selected text.
5. Ask a question about visible screen content.
6. Ask Sahayak to find a relevant interactive element.
7. Give the agent a supported browser task.
8. Review and approve each proposed step.
9. Stop or skip a step at any time.
10. Review the action log.
11. Reuse local profile data and a saved skill.
12. Confirm that protected pages, fields, and actions are blocked by the safety rules.

The final result should feel like a browser assistant that can **read, explain, see, remember, and act - while the user remains in control of every consequential step**.

---

10. Recommended Cursor Execution Order

Use this document as the top-level project specification in Cursor. Build the project in vertical slices rather than generating the entire extension at once.

Recommended order:

1. Extension shell + side panel + settings.
2. Gemini text call.
3. Article extraction and summary.
4. Public YouTube summary.
5. English/Hinglish toggle.
6. Grounded page chat.
7. Context-menu actions.
8. Screenshot-and-ask.
9. API error handling and fallback logic.
10. Interactive element scanner and badges.
11. Structured element selection.
12. Approval-based agent loop.
13. Persistence, dry run, step limits, and action logs.
14. Local profile memory.
15. Saved skills import/export.
16. Safety rails.
17. README, release packaging, and landing page.

For each slice, implement the smallest working version first, test it on real supported pages, and only then move to the next feature.

AI tool

1. Cursor · Download
   https://www.youtube.com/playlist?list=PLfq5z_tJn33A
2. Antigravity IDE
3. VS + https://claude.com/
   https://www.youtube.com/playlist?list=PL0yP9t93Zw_AyNcDz98-Tbi418nlGeoDz

Linkedin → https://www.linkedin.com/in/sakshamarora9575/

Generative AI notes → https://github.com/saksham20189575/generative-ai-notes

Steps

Context Setup

- problemStatement.md → detailed problemStatement of the project we are building
- architecture.md → high level decision that I take while building the project
- implementation-plan.md → phase-wise implementation-plan
- edge-case.md → corner scenarios and edge-cases

Prompts

1. Generate a detailed architecture.md for making this project using @problemStatement.md
2. Generate a phase-wise implementation-plan.md for making this project using @architecture.md @problemStatement.md
3. Generate an edge-case.md which stores all the corner scenarios for the implementation of the project @implementation-plan.md
4. Implement slice 1 of phase1 using the @implementation-plan.md
5. Implement slice 2 from phase1 as per the @implementation-plan.md
6. I can add my API key in .env file if required for slice 2
7. Implement slice 3 of phase1 of @implementation-plan.md

   MANUAL TESTING
8. run the project so that I can test in chrome
9. Copy paste the errors that are coming and fix them
10. Implement slice 4 and slice 5 as per the @implementation-plan.md
11. Implement slice 6 of phase2 as per the @implementation-plan.md
12. Implement slice 7 as per the @implementation-plan.md
13. Implement slice 8 as per the @implementation-plan.md

https://aistudio.google.com/ → GEMINI API KEY
https://groq.com/ → GROQ API Key

Implementation Plan

This plan turns problemStatement.md and architecture.md into build phases. Work one slice at a time. A slice is done only when it runs in Chrome under **Load unpacked** and its checks in this document pass.

The four phases match the four-week spec. The slices inside them match the architecture build order. Safety code starts in Phase 1 as a pure module, and it gates every later path that reads a page or clicks.

How to use this plan

1. Finish the current slice before starting the next.
2. Keep the side panel as the only caller of Gemini and Groq. The content script never sees the API key.
3. Persist approval state in `chrome.storage.session`. Do not leave it in service-worker memory.
4. When a slice touches the panel UI, exercise it in Chrome: open the panel, run the action, and check the empty and error states listed for that slice.
5. Add the unit tests named in the slice in the same change as the code they lock down.

Do not build later features early. In particular, do not add autonomous multi-step execution, login flows, or a backend.

Phase map

Phase	Week	Outcome	Slices
1. Eyes	Week 1	Extension loads, reads a page, summarises articles and public YouTube, switches English / Hinglish	1–5
2. Brain	Week 2	Grounded chat, context-menu actions, screenshot questions, rate-limit fallback	6–9
3. Hands	Week 3	Numbered action map, find-it, approval loop, persistence, practice errands	10–13
4. Memory and ship	Week 4	Profile, skills, full safety wiring, README, zip, landing page	14–17

Phase 1 Eyes

Phase 2 Brain

Phase 3 Hands

Phase 4 Ship

Slice 8 (screenshot) and slice 10 (scanner) depend only on the shell, so they can start once slice 1 is stable. Everything else stays in the order below so each slice has a working caller.

Shared foundations

These files exist from slice 1 and grow in place. Later slices fill functions; they do not invent a second protocol.

File	Created	Responsibility
`manifest.json`	Slice 1	MV3 surface from architecture §5
`src/shared/protocol.ts`	Slice 1	Envelope, command names, record types
`src/shared/storage.ts`	Slice 1	`local` and `session` getters/setters
`src/shared/limits.ts`	Slice 1	Caps: 12k extract, 4k selection, 8 steps, 30s timeout, 20 log sessions

`src/shared/safety-policy.ts`	Slice 1	URL class, field class, consequential names. Pure, no Chrome APIs
`src/shared/models.ts`	Slice 2	Task to model route
`src/ai/client.ts`	Slice 2	One HTTP client for Gemini and Groq
`src/ai/prompts.ts`	Slice 2	Prompt assembly order from architecture §9.5
`src/ai/parse.ts`	Slice 3	Summary, find-it, and agent JSON parsers
`src/background/messaging.ts`	Slice 1	Panel → worker → content router
`src/content/index.ts`	Slice 3	Content-script dispatcher

Default agent allowlist, hard-coded until the settings UI in slice 16:

- `http://localhost:*`
- `http://127.0.0.1:*`

A sensitive host stays blocked even if it is later added to the allowlist.

Phase 1 — The Eyes (Week 1)

The user can load the extension, open the side panel, save a Gemini key, summarise an article, summarise a public YouTube video, and switch the output language.

Slice 1 — Extension shell, panel, settings

Outcome. Chrome loads the unpacked build. The toolbar icon opens the side panel. Settings save a Gemini key in `chrome.storage.local`.

Create

- `package.json`, `vite.config.ts`, `tsconfig.json`
- `manifest.json` with the permissions in architecture §5.1
- `src/background/index.ts` — install handler, `chrome.sidePanel.setPanelBehavior({ openPanelOnActionClick: true })`
- `src/background/messaging.ts` — `panel/ready`, `tab/getActive`
- `src/background/tab-tracker.ts` — active tab id, url, title
- `src/sidepanel/index.html`, `main.tsx`, `app.tsx`
- `src/sidepanel/components/settings.tsx` — Gemini key, optional Groq key
- `src/shared/protocol.ts`, `storage.ts`, `limits.ts`, `safety-policy.ts`
- `.gitignore` including `dist/`, `node_modules/`, `.env*`

- `assets/` icons referenced by the manifest

Steps

1. Scaffold Vite + React + TypeScript + `@crxjs/vite-plugin`. Scripts: `dev`, `build`, `test`.
2. Register the side panel and the empty content-script entry so the manifest is valid. The content script can no-op until slice 3.
3. On panel load, send `panel/ready` and `tab/getActive`. Show the active tab title in the header.
4. Settings form reads and writes `settings`. Empty key shows the missing-key empty state and does not call the network.
5. Add `safety-policy.ts` with `classifyUrl(url)` returning `sensitive`, `allowlisted`, or `normal`, plus `classifyField` and `isConsequentialName`. Ship the denylist and field rules from architecture §8. No UI yet.
6. Add Vitest and the policy tests below.

Tests

- URL matrix: a bank host, a `/checkout` path, `http://127.0.0.1:4173/apply.html`, a normal news URL.
- A bank host that was added to a user allowlist is still `sensitive`.
- Field fixtures: `type=password`, `autocomplete=one-time-code`, `autocomplete=cc-number`, name `otp`, a plain text input.
- Consequential names: Submit, Pay, Send, and a harmless "Next".

Chrome check

- `npm run dev`, Load unpacked on `dist/`.
- Icon opens the panel beside a normal page and shows that page's title.
- Reload the panel and confirm the key is still there.
- Confirm the key is absent from the repo.

Stop. Do not call Gemini in this slice.

Slice 2 — First Gemini text call

Outcome. With a key saved, a text box in the panel sends a prompt to Flash-Lite and renders the reply.

Create

- `src/shared/models.ts` — routes from architecture §9.1. Pin model id strings in this file only.
- `src/ai/client.ts` — non-streaming `generateText` first. Read the key from `settings` at call time. 30s timeout. Return `{ ok, data }` or `{ ok: false, error: { code, message } }`.
- `src/ai/prompts.ts` — `buildPrompt({ task, language, page, profile, history, userText })`. Language and grounding clauses can be stubs until slices 5 and 6.

- `src/sidepanel/components/ask-box.tsx` — temporary until chat replaces it in slice 6.

Steps

1. Implement Gemini `generateContent` for text. No Groq yet.
2. Map missing key to `missing_key`, HTTP errors to `model_unavailable`. Show the sentence from architecture §12, not the raw body.
3. Wire the ask box. This is the plumbing test, not the grounded chat feature.

Chrome check

- Save a key, ask a short question, see a reply in the panel.
- Clear the key, ask again, see the settings prompt and no request.

Slice 3 — Article extraction and structured summary

Outcome. On a normal article, Summary shows a one-line TL;DR, five key points, and a next step.

Create

- `src/content/extract.ts` — Readability on a cloned document; fallback to clipped `innerText`; `kind` is `article`, `youtube`, or `generic`
- `src/content/index.ts` — handle `page/extract`
- `src/ai/parse.ts` — `parseSummary`
- `src/sidepanel/components/summary.tsx`
- Summary mode in the panel tabs (Chat and Act can be disabled placeholders)

Steps

1. Worker forwards `page/extract` only after `classifyUrl`. Sensitive URLs return `sensitive_page` and do not call the model.
2. Cap extracted text at 12,000 characters, keeping the start and the end.
3. Detect YouTube watch URLs and set `kind: "youtube"` without sending the transcript anywhere yet.
4. Call Flash-Lite with the summary schema. Parse before render. If the model returns more than five points, keep five.
5. If Readability returns nothing, still summarise the generic extract and label it as a generic page.

Tests

- `parseSummary` accepts a valid object and rejects missing `tldr`.
- Extract truncation keeps both ends. Use a fixture string in unit tests; DOM extraction is checked manually.

Chrome check

- Summarise a news article. Navigation, ads, and footer text should not dominate the points.

- Open a sensitive URL, run Summary, and confirm the blocked message. Do not verify by watching the network only; the panel must show `sensitive_page`.

Slice 4 — Public YouTube summary

Outcome. A public watch URL produces the same summary shape plus chapter notes. Each timestamp is a link that seeks the video.

Steps

1. When `kind === "youtube"`, skip Readability text as the model context. Send the watch URL as Gemini file input on the Flash route from `models.ts`.
2. Extend the parser with optional `chapters: { timestamp, note }[]`.
3. Convert `mm:ss` and `h:mm:ss` into `&t=` seconds on the same video id.
4. If Gemini rejects the video, show that this video cannot be summarised. Do not scrape captions.

Tests

- Timestamp conversion: `1:02` → 62, `1:02:03` → 3722.
- A non-YouTube summary does not render a chapter list.

Chrome check

- One public watch URL: summary plus at least one working timestamp link.
- An unsupported or private URL: the error state, panel still usable.

Slice 5 — English / Hinglish toggle

Outcome. The header toggle sets `settings.language`. The next summary uses that language. Older messages stay as rendered.

Steps

1. Add the toggle. Values: `en`, `hinglish`.
2. Thread `language` through `buildPrompt`. Hinglish means Hindi in Latin script mixed with English.
3. Re-run summary after a toggle and confirm the new reply follows the new language.

Chrome check

- Summarise the same article in both languages.
- Reload the panel and confirm the choice persisted.

Phase 1 exit

Week 1 acceptance in the problem statement is met:

- Unpacked load works.
- Side panel opens beside a normal page.
- Key is entered in settings, not hard-coded.
- Article summary has TL;DR, five points, and a next step.

- A public YouTube link summarises, with timestamp links when chapters exist.
- English / Hinglish switching works.

Selected-text explain is completed in slice 7, which is the first week-2 slice that needs the chat surface. If Phase 1 must demo explain before Phase 2 starts, point the slice 2 ask box at `page/selection` as a temporary path and replace it in slice 7. Do not build a second explain pipeline.

Phase 2 — The Brain (Week 2)

The panel can chat with the current page, act on a selection from the context menu, answer a question about a cropped screenshot, and survive model errors.

Slice 6 — Grounded page chat

Outcome. Chat answers from the current page, says when the answer is not on the page, keeps history for the active tab, and streams tokens.

Create

- `src/sidepanel/components/chat.tsx`
- Streaming method on `ai/client.ts` (`generateContentStream` or equivalent SSE)

Steps

1. Replace the slice 2 ask box with the chat composer.
2. On each send, refresh `page/extract`. Refuse sensitive URLs before the stream starts.
3. Assemble prompts in the order from architecture §9.5. Cap history at the last 8 turns.
4. Store the transcript in React state keyed by `tabId`. A tab change clears it and shows the new title.
5. System prompt: answer only from the supplied context; if it is missing, say the answer is not on this page. Surface that as `off_page` when the model declines.
6. Stream into the latest assistant bubble. On failure, leave the user message in place and show the error code's sentence.

Chrome check

- Ask something answered by the article. The reply matches the page.
- Ask something absent from the page. The reply says it is not on the page.
- Switch tabs. The transcript resets.
- Start a stream and confirm tokens appear before the full reply finishes.

Slice 7 — Context-menu actions

Outcome. Selecting text and choosing Explain this, Translate, or Draft a reply opens the panel and shows the result.

Create

- `src/background/context-menus.ts`

- `src/content/selection.ts` — `page/selection` for the panel's own explain path
- Pending-intent helpers in `storage.ts`

Steps

1. Create three selection context menus on install.
2. On click, write `pendingIntent` (`kind`, `selectionText`, `tabId`, `url`, `createdAt`) to `chrome.storage.session`, then open the side panel.
3. `panel/ready` returns the intent. The panel runs the matching prompt and clears the key. Drop intents older than 2 minutes.
4. Explain simplifies the selection. Translate uses `settings.translateLanguage` (default `hi`) and does not change `settings.language`. Draft a reply writes a reply in the active output language.
5. Render results in the chat thread so there is one result surface.
6. Also support Explain from the panel using `page/selection` when the user has a selection and the panel is already open.

Chrome check

- Each menu item, with the panel closed, opens the panel and shows a result.
- Translate stays on the translate language after the global toggle is flipped to the other output language.
- A sensitive page selection does not produce a model call.

Slice 8 — Screenshot and ask

Outcome. The user captures the visible tab, crops a region, asks a question, and gets an answer about that image.

Create

- `src/sidepanel/components/cropper.tsx`

Steps

1. Add "Ask about this screen" on Chat. The question can be typed before capture.
2. Worker handles `capture/visible` via `chrome.tabs.captureVisibleTab`. Refuse sensitive URLs before capture.
3. Draw the PNG on a canvas. A drag sets the crop. Skip uses the full frame.
4. Export JPEG at quality about 0.7 with the longest edge at most 1280px. Send it on the Flash vision route. No Groq path.
5. Drop the data URL when the answer arrives, the user cancels, or the tab changes. Do not write it to storage or the action log.

Chrome check

- Crop a chart or error message and ask a question that needs the pixels.
- Cancel the crop and confirm nothing is sent.
- On a sensitive URL the capture button is hidden or the click returns `sensitive_page` without a capture call.

Slice 9 — Errors, retry, and fallback

Outcome. HTTP 429 and 5xx do not crash the panel. Text tasks fall back to Groq. Image and YouTube tasks show a clear error instead.

Steps

1. In `ai/client.ts`: on 429, honor `Retry-After` up to 8 seconds, retry the same route once, then follow the fallback column in architecture §9.1.
2. On 5xx or network failure, retry once, then fall back.
3. Groq is text-only. Skip it when the route has an image, a YouTube file URI, or a sensitive-page flag.
4. Optional Groq key in settings. If Gemini fails and no Groq key is stored, return `rate_limited` or `model_unavailable`.
5. Panel copy while waiting: "Sahayak is waiting for the free model."
6. Invalid JSON is not handled here except for a client hook used in slice 11: re-ask once with the parser error, then escalate to Flash once.

Tests

- A fake fetch that returns 429 then 200 calls the primary twice and not Groq.
- A fake fetch that returns 429 twice calls Groq for a text task and does not call Groq for an image task.

Chrome check

- Force a bad key and confirm the panel shows `model_unavailable` and still accepts a new prompt.
- With a Groq key saved, confirm a text fallback path is wired (use a temporary forced-fallback flag in dev only, and remove it before release).

Phase 2 exit

- Chat stays on the page and admits missing answers.
- Explain, Translate, and Draft a reply work from the context menu.
- A cropped screenshot question returns an answer.
- Rate limits and model errors do not crash the extension.